

# Rapport sur le Problème de Chargement de Ferry

## CSI2510- Structure de Donnée et Algorithme

Automne 2024

École de Génie Électrique et Science Informatique

Université d'Ottawa

Ahansal,Niamatollah,300386816

Partie I:BackTracking avec mémorisation en utilisant Big Table :

### Résumé de la validité du code:

L'implémentation utilisant BigTable a réussi tous les cas de test et a été acceptée par OnlineJudge . La solution a utilisé un tableau “state” bidimensionnel pour enregistrer les états visités et éviter les calculs redondants.

### Résultats du Juge en Ligne:

- **Capture d'écran :**

[#56425742](#) | [Niamatollah's solution for \[UVA-10261\]](#)

| Status   | Time  | Length | Lang       | Submitted           | Open                     | Share text ?             | RemoteRunId |
|----------|-------|--------|------------|---------------------|--------------------------|--------------------------|-------------|
| Accepted | 810ms | 3828   | JAVA 1.8.0 | 2024-11-27 18:34:24 | <input type="checkbox"/> | <input type="checkbox"/> | 29999763    |

- **Temps d'exécution** : Le temps d'exécution pour cette approche était de **810ms** pour de grandes entrées.

### Observations:

1. **Utilisation de l'espace** : La solution consomme beaucoup de mémoire à cause du tableau "State" de dimensions  $[n+1][L+1]$ , où  $n$  est le nombre de voitures et  $L$  la longueur du ferry en centimètres.
2. **Complexité temporelle** : Bien que cette approche évite des calculs redondants, elle explore encore un grand nombre de possibilités, ce qui la rend plus lente pour de grandes entrées.

## Partie II:BackTracking avec mémorisation en utilisant HashTable:

### Résumé de la validité du Code:

L'optimisation avec une table de hachage a également passé tous les tests et a été acceptée par le juge en ligne. Cette approche a remplacé la grande table d'état par une table de hachage, améliorant ainsi considérablement les performances.

### Résultats du Juge en Ligne

- **Capture d'écran** :

[#56426084](#) | Niamatollah's solution for [\[UVA-10261\]](#)

| Status   | Time  | Length | Lang       | Submitted           | Open                     | Share text ?             | RemoteRunId |
|----------|-------|--------|------------|---------------------|--------------------------|--------------------------|-------------|
| Accepted | 350ms | 4118   | JAVA 1.8.0 | 2024-11-27 19:24:11 | <input type="checkbox"/> | <input type="checkbox"/> | 29999932    |

- **Temps d'exécution** : Le temps d'exécution était de **350 ms**, soit 57 % plus rapide que l'implémentation avec la table de hachage .

### Observations:

- **Efficacité spatiale** : En limitant le stockage aux seuls états nécessaires de manière dynamique, la table de hachage a optimisé l'utilisation de la mémoire, réduisant ainsi les exigences en termes d'espace.

- **Efficacité temporelle** : Grâce à une fonction de hachage performante, les opérations de recherche et d'insertion des états ont été réalisées de manière rapide, éliminant efficacement les calculs redondants.

## Partie III:Détails sur la Conception de L'Implémentation de la Table de Hachage

### Fonction de Hachage:

La fonction de hachage utilisée est :

$\text{hashFunction}(\text{key}, \text{value}) = (\text{key} + \text{value}) \% \text{hashTableSize}$ .

- **Key ( $\text{currk}+1$ )** Représente l'état pour la prochaine voiture à examiner. L'utilisation de  **$\text{currk}+1$**  assure que chaque état est identifié de façon unique en fonction du nombre de voitures déjà placées, augmentant de un à chaque étape, et élimine ainsi toute ambiguïté dans le processus récursif. Valeur ( **$\text{currS}$** ) : Représente l'espace restant du côté gauche (port) du ferry.

### Hypothèses et Limitations:

**Hypothèse Principale** : La fonction de hachage est efficace car la nature unique de la clé ( $\text{currK}+1$ ) et la valeur ( $\text{currS}$ , souvent distincte) minimise les collisions dans le scénario du problème.

**Limitation** : Si les collisions devenaient fréquentes, il faudrait gérer Key et Value comme des variables distinctes dans la table, ce qui nécessiterait :

- **Gestion séparée** : Un suivi séparé de la clé et de la valeur dans une structure de données adaptée.
- **Gestion Optimisée des États** : Garantir que leur combinaison (représentant un état) est efficacement suivie et récupérée.

### Gestion des Collisions:

En cas de collision, le sondage **linéaire** est utilisé. Cette méthode est particulièrement efficace avec une table de grande taille, car elle garantit que tous les emplacements de la table sont accessibles.

### Taille de la Table de Hachage:

La taille de la table de hachage est définie comme suit :

$\text{hashTableSize} = n + L$

Cela garantit une capacité suffisante pour minimiser les collisions et maximiser les performances.

**Initialisation:** La table de hachage est initialisée à -1, signalant que les emplacements n'ont pas été visités. Cela simplifie la vérification des états.

### Améliorations des Performances:

- **Réduction des Collisions :** La combinaison de  $\text{currK}+1$  avec  $\text{currS}$  garantit des collisions rares et une gestion efficace.
- **Optimisation de la Mémoire :** Contrairement à une grande table, la table de hachage n'utilise de mémoire que pour les états réellement nécessaires.
- **Accès Rapide :** Le hachage permet une recherche quasi instantanée des états, réduisant significativement le temps d'exécution global.

### Comparaison des Approches:

|                             | Grande Table          | Table de Hachage                    |
|-----------------------------|-----------------------|-------------------------------------|
| Temps d'exécution           | 810ms                 | 350ms                               |
| Complexité Mémoire          | $O(n \times L)$       | $O(\text{tailleTableHachage})$      |
| Complexité d'implémentation | Modérée               | Plus élevée                         |
| Gestion des Collisions      |                       | Sondage linéaire                    |
| Évolutivité                 | Limitée par la taille | Performante pour de grandes entrées |

Conclusion:

L'utilisation d'une table de hachage apporte des gains considérables en mémoire et en temps par rapport à une grande table. La fonction de hachage fonctionne très bien ici grâce à un faible nombre de collisions. Cependant, si les collisions devenaient plus fréquentes, il faudrait envisager de séparer la clé et la valeur pour mieux gérer les états. Cela montre à quel point il est essentiel d'adapter les techniques de hachage aux besoins spécifiques du problème pour obtenir les meilleures performances possibles.